

Module 6 : Javascript Essential and Advanced

Q - 1 : What is JavaScript? Explain the role of JavaScript in web development.

A - 1 : JavaScript is a high-level, interpreted programming language used to make web pages interactive and dynamic.

- Role in web development:

- Adds interactivity (buttons, forms, animations)
- Updates content without reloading page
- Works with HTML (structure) and CSS (style)

Q - 2: How is JavaScript different from other programming languages like Python or Java?

A - 2 :

Feature	JavaScript	Python	Java
Execution	Browser + Server	Mostly Server	JVM
Typing	Dynamic	Dynamic	Static
Use	Web	AI, Data	Enterprise apps
Syntax	Flexible	Simple	Strict

Q - 3 : Discuss the use of <script>tag in HTML. How can you link an external JavaScript file to an HTML document?

A - 3 : Used to include JavaScript in HTML.

Internal JS :

```
</>HTML
```

```
<script>  
alert("Hello");  
</script>
```

External JS :

```
</>HTML  
  
<script src="app.js"></script>
```

Q - 4 : What are variables in JavaScript? How do you declare a variable using var, let, and const?

A - 4 : Variables in JavaScript are used to store data values that can be used and modified in a program. They act like containers for holding information such as numbers, strings, or objects.

1. var	2. let	3. const
var x = 10; var x = 20; // allowed x = 30; // allowed	let y = 10; y = 20; // allowed // let y = 30; not allowed in same scope	const z = 10; // z = 20; not allowed

Q - 5: Explain the different data types in JavaScript. Provide examples for each.

A - 5 : JavaScript data types define the type of data a variable can hold. They are mainly divided into Primitive and Non-Primitive (Reference) types.

1. Primitive Data Types :	
(a) Number	let a = 10;
(b) String	let name = "Vrutti";
(c) Boolean	let isActive = true;
(d) Undefined	let x; console.log(x); // undefined

1. Primitive Data Types :	
(e) Null	let y = null;
(f) BigInt	let big = 12345678901234567890n;
(g) Symbol	let sym = Symbol("id");

2. Non-Primitive (Reference) Data Types	
(a) Object	let person = { name: "Vrutti", age: 20 };
(b) Array	let arr = [1, 2, 3, 4];
(c) Function	function greet() { console.log("Hello"); }

Q - 6: What is the difference between undefined and null in JavaScript?

A - 6 :

1. undefined	2. null
A variable that is declared but not assigned any value	A variable that is intentionally assigned an empty value
It is assigned automatically by JavaScript	It is assigned by the programmer
Example : let x; console.log(x); // undefined	Example : let y = null;

Q - 7: What are the different types of operators in JavaScript? Explain with examples.

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators

A - 7 :

Arithmetic operators (Used for mathematical calculations.)		
Operator	Meaning	Example
+	Addition	$10 + 5 = 15$
-	Subtraction	$10 - 5 = 5$
*	Multiplication	$10 * 5 = 50$
/	Division	$10 / 5 = 2$
%	Modulus (remainder)	$10 \% 3 = 1$

Assignment Operators (Used to assign values to variables.)		
Operator	Meaning	Example
=	Assign value	$x = 10$
+=	$x = x + 5$	$x += 5$
-=	$x = x - 5$	$x -= 5$
*=	$x = x * 2$	$x *= 2$

Comparison Operators (Used to compare two values and return true or false.)		
Operator	Example	Result
==	5 == "5"	true
===	5 === "5"	false
!=	5 != 3	true
>	10 > 5	true
<	10 < 5	false

Logical Operators (Used to combine multiple conditions)		
Operator	Meaning	Example
&&	AND (both true)	true && false = false
 	OR	true if at least one condition is true
!	NOT (reverse)	!true = false

Q - 8: What is the difference between ==and ===in JavaScript?

A - 8 :

1. **== (Loose Equality) :**

- Compares only values
- Automatically converts (type coercion) data types if needed
- Example : console.log(5 == "5"); // true

2. === (Strict Equality) :

- Compares value and data type
- Does **not** perform type conversion
- Example : `console.log(5 === "5"); // false`

Q - 9: What is control flow in JavaScript? Explain how if-else statements work with an example.

A - 9 : Control flow refers to the order in which statements are executed in a program.

if-else Statements :

- The if-else statement is used to make decisions.
- It executes different blocks of code depending on whether a condition is true or false.

Example :

```
let marks = 75;

if (marks >= 50) {
    console.log("Pass");
} else {
    console.log("Fail");
}
```

Q - 10: Describe how switch statements work in JavaScript. When should you use a switch statement instead of if-else?

A - 10 : A switch statement is used to execute one block of code from multiple options based on the value of a variable or expression.

How it Works :

- The switch evaluates an expression once
- It compares the result with each case
- When a match is found, that block of code runs
- break is used to stop execution after a match

- default runs if no case matches

Use a switch statement when:

- You are checking multiple values of the same variable
- Conditions are based on equality (== or ===)
- You want cleaner and more readable code

Q - 11 : Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.

A - 11 : Loops are used to execute a block of code repeatedly until a certain condition is met.

1. for Loop : Used when the number of iterations is known in advance.

Example :

```
for(let i = 0; i < 5; i++){  
    console.log(i);  
}  
  
//Prints numbers from 0 to 4
```

2. while Loop : Used when the number of iterations is not known beforehand.

Example :

```
let i = 0;  
while(i < 5){  
    console.log(i);  
    i++;  
}  
  
//Runs while the condition is true
```

3. do-while Loop : Executes the code at least once, even if the condition is false.

Example :

```
let i = 0;

do{
  console.log(i);
  i++;
}while(i < 5);

// Runs first, then checks condition
```

Q - 12: What is the difference between a while loop and a do-while loop?

A - 12 :

Feature	while	do-while
Condition Check	Before loop	After loop
Execution	May not run	Runs at least once
Use Case	When condition must be true first	When at least one execution is needed

Q - 13: What are functions in JavaScript? Explain the syntax for declaring and calling a function.

A - 13 : Functions are reusable blocks of code that perform a specific task. They help in avoiding repetition and make code more organized and readable .

Syntax for Declaring a Function :	Syntax for Calling a Function :
<pre>function functionName(parameters){ // code to execute }</pre>	<pre>FunctionName();</pre>

Q - 14: What is the difference between a function declaration and a function expression?

A - 14 :

1. Function Declaration :

- Defined using the function keyword with a name
- Can be called before it is defined (hoisting)

```
function greet(){  
    console.log("Hello");  
}
```

2. Function Expression :

- Function is assigned to a variable
- Can be anonymous (no name)
- Cannot be called before definition

```
const greet = function(){  
    console.log("Hello");  
};
```

Q - 15 : Discuss the concept of parameters and return values in functions.

A - 15 : parameters and return values are used to pass data into a function and get results back.

1. Parameters : Parameters are the **inputs** given to a function when it is defined.

- They allow the function to work with different values
- Written inside the parentheses ()
- Input to function
- **Example :**

```
function greet(name){  
    console.log("Hello " + name);  
}
```

greet("Vrutti");

Output: **Hello Vrutti**

2. Return Values : A return value is the **output** that a function sends back after execution.

- Defined using the return keyword
- Stops the function execution
- Output from function
- **Example :**

<pre>function add(a, b) { return a + b; }</pre>	<pre>let result = add(5, 3); console.log(result); // 8</pre>
---	--

Q - 16: What is an array in JavaScript? How do you declare and initialize an array?

A - 16 : An array in JavaScript is a data structure used to store multiple values in a single variable.

- These values can be of the same or different data types.
- Each value in an array is stored at an index, starting from 0.

Declaring and Initializing an Array :

1. Using Square Brackets (Most Common Method) :

```
let arr = [1, 2, 3, 4];
```

2. Using Array Constructor :

```
let arr = new Array(1, 2, 3, 4);
```

3. Empty Array and Adding Values Later :

```
let arr = [ ];
```

```
arr[0] = 10;
```

```
arr[1] = 20;
```

4. Example with Different Data Types :

```
let data = ["Vrutti", 20, true];
```

Q - 17: Explain the methods push(), pop(), shift(), and unshift() used in arrays.

A - 17 :

Method	Action	Position
push()	Add element	End
pop()	Remove element	End
shift()	Remove element	Beginning
unshift()	Add element	Beginning

Q - 18 : What is an object in JavaScript? How are objects different from arrays?

A - 18 : An object is a data type used to store multiple values in the form of key-value pairs.

- Each value is accessed using a key (property name)
- Useful for representing real-world entities

Example :

```
let person = {
  name: "Vrutti",
  age: 20,
  isStudent: true
};
```

Difference Between Object and Array :		
Feature	Object	Array
Structure	Key-value pairs	Ordered list of values
Access	Using keys	Using index (0,1,2...)
Use	Complex data (person, product)	List of items
Syntax	{ }	[]

Q - 19 : Explain how to access and update object properties using dot notation and bracket notation.

A - 19 :

1. Dot Notation (.) : Used when the property name is known and simple (no spaces or special characters).

Accessing Properties:	Updating Properties:
<pre>let person = { name: "Vrutti", age: 20 }; console.log(person.name); // Vrutti</pre>	<pre>person.age = 21; console.log(person.age); // 21</pre>

2. Bracket Notation ([]) : Used when Property name has spaces or special characters and Property name is stored in a variable.

Accessing Properties:	Updating Properties:
<pre>console.log(person["name"]); // Vrutti</pre>	<pre>person["age"] = 22; console.log(person["age"]); // 22</pre>

Q - 20 : What are JavaScript events? Explain the role of event listeners.

A - 20 : JavaScript events are actions or occurrences that happen in the browser, usually triggered by the user or the system.

Examples of Events:

- **click** → when a button is clicked
- **mouseover** → when mouse moves over an element
- **keydown** → when a key is pressed
- **load** → when a page finishes loading

Role of Event Listeners : An **event listener** is used to **detect an event and execute a function** when that event occurs.

- It “listens” for a specific event
- When the event happens, it runs a callback function

Example :

```
document.getElementById("btn")
    .addEventListener("click", function(){
        alert("Button clicked!");
    });
```

Q - 21: How does the addEventListener() method work in JavaScript? Provide an example.

A - 21 : The addEventListener() method is used to **attach an event handler to an HTML element**.

- It listens for a specific event and executes a function when that event occurs.

Syntax : element.addEventListener(event, function, useCapture);

How it Works :

1. Select an HTML element
2. Attach an event listener to it
3. When the event happens, the function runs automatically

Example :

```
let button = document.getElementById("btn");

    button.addEventListener("click", function(){
        console.log("Button was clicked!");
    });
```

Q - 22 : What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?

A - 22 : The DOM (Document Object Model) is a programming interface that represents an HTML document as a tree structure of objects.

- Each element in HTML (like <h1>, <p>, <div>) becomes an **object (node)** in the DOM.

JavaScript uses the DOM to **access and manipulate HTML elements** :

1. **Selecting Elements** : `document.getElementById("id");`
2. **Changing Content** : `document.getElementById("demo").innerHTML = "Hello World";`
3. **Changing Styles** : `document.getElementById("demo").style.color = "red";`
4. **Adding/Removing Elements** : `let p = document.createElement("p");`
`document.body.appendChild(p);`
5. **Handling Events** :
`document.getElementById("btn").addEventListener("click", function(){`
`alert("Clicked!");`
`});`

Q - 23 : Explain the methods `getElementById()`, `getElementsByClassName()`, and `querySelector()` used to select elements from the DOM.

A - 23 : These methods are used to select HTML elements from the DOM so that JavaScript can manipulate them.

1. `getElementById()` :

- Selects a single element by its id
- Returns one element
- Used when the element has a **unique id**

Example :

```
let element = document.getElementById("demo");
```

```
element.innerHTML = "Hello";
```

2. `getElementsByClassName()` :

- Selects elements by **class name**
- Returns a **collection (HTMLCollection)** of elements
- Multiple elements can have the same class

Example :

```
let items = document.getElementsByClassName("box");  
items[0].style.color = "red";
```

3. `querySelector()` :

- Selects the **first matching element**
- Uses **CSS selectors**
- Can select using:
 - **#id**
 - **.class**
 - **tag**

Example :

```
let element = document.querySelector(".box");  
element.style.backgroundColor = "yellow";
```

Q - 24: Explain the `setTimeout()` and `setInterval()` functions in JavaScript. How are they used for timing events?

A - 24 : These functions are used to control timing of code execution, allowing actions to happen after a delay or repeatedly over time.

1. `setTimeout()` :

- Executes a function once after a specified delay
- Time is given in milliseconds (1000 ms = 1 second)

Syntax : setTimeout(function, delay);

Example :

```
setTimeout(function(){  
    console.log("Hello after 2 seconds");  
}, 2000);
```

// Runs only **one time** after 2 seconds

2. setInterval() :

- Executes a function repeatedly at fixed intervals.

Syntax : setInterval(function, interval);

Example :

```
setInterval(function(){  
    console.log("Running every 1 second");  
}, 1000);
```

// Runs **again and again every 1 second**

Stopping the Timer :

You can stop these functions using:

- clearTimeout()
- clearInterval()

Example :

```
let id = setInterval(function(){
    console.log("Hello");
}, 1000);

clearInterval(id);
```

Q - 25 : Provide an example of how to use setTimeout() to delay an action by 2 seconds.

A - 25 : The setTimeout() function is used to **delay the execution of code** for a specified time (in milliseconds).

Example :

```
setTimeout(function(){
    console.log("This message appears after 2 seconds");
}, 2000);

// After 2 seconds, the message will be printed in the console.
```

Q - 26: What is error handling in JavaScript? Explain the try, catch, and finally blocks with an example.

A - 26 : Error handling is the process of **detecting and managing errors** in a program so that it **does not crash** and continues to run smoothly.

1. try Block :

- Contains code that **may cause an error**
- JavaScript attempts to execute this code

2. catch Block :

- Executes if an **error occurs in try**
- It handles the error and prevents program crash

3. finally Block :

- Executes **always**, whether an error occurs or not
- Used for cleanup operations

Example :

```
try {  
    let result = x + 10; // x is not defined (error)  
} catch (error) {  
    console.log("Error occurred:", error.message);  
} finally {  
    console.log("Execution completed");  
}
```

Q - 27 : Why is error handling important in JavaScript applications?

A - 27 : Error handling is important in JavaScript applications because it helps to **manage runtime errors** and ensures that the program runs smoothly without crashing.

Key Reasons :

1. Prevents Application Crashes :

- Without error handling, a single error can stop the entire program.
- With error handling, the program can continue running.

2. Improves User Experience :

- Instead of showing confusing errors, you can display friendly messages to users.

3. Helps in Debugging :

- Error handling provides information about errors, making it easier for developers to identify and fix issues.

4. Ensures Smooth Execution :

- Ensures Smooth Execution

5. Handles Unexpected Situations :

- Helps manage issues like:
 - Invalid user input
 - Network errors
 - Missing data